Assignment No 4

Aim: Apply Naïve Bayes classification algorithms.

Theory:

Naïve Bayes is a supervised machine learning algorithm used mainly for **classification problems**. It is based on **Bayes' Theorem**, which calculates the probability of a particular class based on the available features.

The algorithm is called **"Naïve"** because it assumes that all input features are **independent of each other given the class**. Although this assumption may not always be true in real-world datasets, Naïve Bayes performs very well for many classification tasks.

For example, suppose we want to classify an email as **Spam** or **Not Spam**. Features such as the presence of certain words, number of links, and email length can be used to predict the class of the email.

Bayes' Theorem

The basic formula used by Naïve Bayes is:

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

Where:

- **P(C|X)** = Probability of class C given the features X
- **P(X|C)** = Probability of observing the features given class C
- **P(C)** = Prior probability of the class
- **P(X)** = Probability of observing the features

The algorithm calculates the probability of each possible class and assigns the data point to the class having the **highest probability**.

Types of Naïve Bayes

Different versions of Naïve Bayes are used depending on the type of data:

1. **Gaussian Naïve Bayes**
   Used when the numerical features are assumed to follow a Gaussian/normal distribution.
2. **Multinomial Naïve Bayes**
   Commonly used for text classification, such as document or spam classification.

3. **Bernoulli Naïve Bayes**
   Used when features are binary, such as 0/1 or Yes/No.

Working of Naïve Bayes

The basic steps are:

1. Collect and prepare the dataset.
2. Separate the input features and target class.
3. Divide the dataset into training and testing data.
4. Select an appropriate Naïve Bayes algorithm.
5. Train the model using the training dataset.
6. Calculate the probability of each class.
7. Predict the class of the test data.
8. Evaluate the model using metrics such as accuracy, precision, recall, and F1-score.

Advantages

- Simple and easy to implement.
- Fast training and prediction.
- Works well with large datasets.
- Requires relatively little training data.
- Particularly effective for text classification.
- Can handle multiple classes.

Limitations

- Assumes independence between features.
- Performance may decrease when features are strongly related.
- Probability estimates may not always be accurate.
- Continuous features may require appropriate assumptions or preprocessing.

Applications

Naïve Bayes is widely used in:

- Spam email detection.

- Sentiment analysis.
- Text classification.
- News classification.
- Document categorization.
- Medical diagnosis.
- Recommendation systems.
- Customer classification.

Conclusion

Naïve Bayes is a simple and efficient supervised classification algorithm based on Bayes' Theorem. It uses probability to classify data into different categories and is especially useful for applications involving text and categorical data.

Code:

```
import pandas as pd

from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_wine()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


nb_classifier = GaussianNB()
```

```
nb_classifier.fit(X_train_scaled, y_train)

y_pred = nb_classifier.predict(X_test_scaled)


accuracy = accuracy_score(y_test, y_pred)

conf_matrix = confusion_matrix(y_test, y_pred)

report = classification_report(y_test, y_pred)

print("Accuracy:", accuracy)

print("Confusion Matrix:\n", conf_matrix)

print("Classification Report:\n", report)
```

Output

